

Programming 1

Orga: Eval AIN



Programming 1
(AIN-B):

[https://eva.th-deg.de/
evasys/online.php?
typ=html&user_tan=Z
KZ7J](https://eva.th-deg.de/evasys/online.php?typ=html&user_tan=ZKZ7J)

Orga

- We all have a solid understanding of
 - variables, types and collections?
 - Loops, conditions
 - Functions & modules
- Any questions so far?

Calendar

6./7.10.	Intro, Orga, Hello, World!	8./9.12.	lectures
13./14.10.	Variables, basics	15./16.12.	mid-term
20.21./10.	Collections, Lists, ...	22./23.12.	lectures
27./28.10.	Conditions and Loops	12./13.1.	lectures
3./4.11.	Functions and Modules	18./19.1.	recap
10./11.11.	Classes and Methods	from 26.1.	exams...
17./18.11.	UI + TK basics		
24./25.11.	Spaceship		
01./02.12.	Invaders and Collisions		

Classes and Methods

Objects

- How do we „create“ our spaceship for the mission to mars?

Objects

- How do we „create“ our spaceship for the mission to mars?
 - A set of variables for the details?
 - Name = „name“
 - Crewmembers = [„name“, „name2“, ...]
 - Fuel_level = 100
 - Maybe grouping the variables as a list?

Objects

- How do we „create“ our spaceship for the mission to mars?
 - A dictionary describing it?
 - Spaceship = {
 - name: „name“,
 - crewmembers: [„name“, „name2“, ...],
 - fuellevel: 100

Methods

- How do we „create“ actions for our spaceship?
 - `def add_crewmember(spaceship, name):`
 `pass`
 - `def refuel(spaceship, amount):`
 `pass`

Object Oriented Programming

- How about we tune our thinking from procedural programming (functions, sequential flow, ...) ?
 - Let's think of an object that can perform actions:
 - Spaceship: holding all relevant information itself
 - Provide actions e.g. refuel, takeoff, ...
- let's look into object oriented programming

Object Oriented Programming

- OOP is a way of organizing code by grouping data(variables) and functions together into objects
- Instead of just writing functions and variables separately , OOP lets us create blueprints called CLASSES.
- Classes can be defined as blueprints for a certain category of objects.
- Making an object from a class is called instantiation, a user works with instances of a class.

```
# Spaceship class
class Spaceship:
    def __init__(self, name):
        self.name = name
        self.fuel = 100
        self.speed = 0

    def accelerate(self):
        if self.fuel > 0:
            self.speed += 10
            self.fuel -= 5
            print("Accelerating:", self.speed)

# Crew class
class CrewMember:
    def __init__(self, name, role):
        self.name = name
        self.role = role

    def perform_task(self, task):
        print(self.name, "is performing", task)
```

Why OOP?

- OOP :
 - Think of it as describing real-world objects and their interactions
 - Code is organized around objects that contain both data and methods
 - Deals with entities
- Procedural programming :
 - Think of it as a recipe or step by step instruction
 - Code is organized as a sequence of functions or “procedures”
 - Data and functions are separate

Why OOP?

- Problems with Procedural Programming:
 - Code Organization issues
 - Functions scattered everywhere
 - Hard to find related code
 - Difficult to maintain large projects
 - Data security
 - Global variables can be modified anywhere
 - No control over who has access to data
 - Easy to accidentally break things

Why OOP?

- Code Reusability Challenges:
 - Hard to reuse code in different contexts
 - Lots of code duplication
 - Changes in one place affect many others
- Complexity Management:
 - As projects grow, becomes harder to understand
 - Relationships between functions unclear
 - Debugging becomes a nightmare
- **ALL THE ABOVE SHORTCOMINGS CAN BE FIXED WHEN USING OOP**

Classes

- Creating a class :
 - Let's say we create a class Dog, this doesn't represent a particular dog. Rather it represents any dog.
 - We know that a pet dog usually has the following attributes:
 - Name
 - Age
 - Can do tricks such as sit, roll over

Example of a Class

```
class Dog:
    def __init__(self, name, age): #Constructor, this runs automatically when a new object is created
        self.name = name #self keyword represents the current object (instance) of the class
        self.age = age #self makes sure the attributes belong to the specific object

    def sit(self):
        print(f"{self.name} sat down")

dog1 = Dog("Johnny", 4)
dog1.sit() #OUTPUT : Johnny sat down
```

Classes

- The `__init__` method runs **automatically** and allows the user to set the initial values. When creating a dog we see that name and age need to be provided.
- The values are assigned to the respective instance variables of the object by the **constructor** using the keyword “self”.
- dog1 here is the **instance** of the class Dog.
It is an object created from the blueprint Dog.
The object dog1 has the attributes :
 - Name : Johnny
 - Age : 4

Class variables

- Variables defined in a class that are shared across all instances of a class.
- This means that every instance or object created from the class has the same value of this variable
- Example:
 - In our dog class if we set a variable called species = “Canine”
 - Every dog object we create after this would always have the species as Canine

Instance Variables

- Variables defined in a class that are unique to each instance or object of a class.
- These are defined within the `__init__` method or other instance methods unlike the Class variables which are defined outside.
- Each object maintains its own instance variables which would differ from one object to the other
- Example:
 - Every pet dog we create can have a different name and different age
 - These attributes, name, age are all instance variables

Example

```
class Dog:
    species = "Canine" #class variable (shared)
    def __init__(self, name, age): #Constructor, this runs automatically when a new object is created
        self.name = name #Instance variables(unique for every dog instance)
        self.age = age
    def sit(self):
        print(f"{self.name} sat down")

dog1 = Dog("Buddy",3)
dog2 = Dog("Luna",5)
print(dog1.species , dog2.species) #Output : Canine Canine
print(dog1.name, dog2.name) #Output : Buddy Luna
```

Accessing Attributes

- To access the attributes of an instance a user uses the dot notation. To access the name of the dog :
 - dog1.name

```
class Dog:
    def __init__(self, name, age): #Constructor, this runs automatically when a new object is created
        self.name = name #self keyword represents the current object (instance) of the class
        self.age = age #self makes sure the attributes belong to the specific object

    def sit(self):
        print(f"{self.name} sat down")

dog1 = Dog("Johnny", 4)
dog1.sit() #OUTPUT : Johnny sat down
print(dog1.name) #OUTPUT : Johnny
```

Polymorphism

- Polymorphism → one name, many forms
- It allows different classes to have methods with the same name but different behavior.

```
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

animals = [Dog(), Cat()]
for animal in animals:
    print(animal.speak())
```

Polymorphism

- In our example we can see that dogs and cats make a different sound.
- With the help of Polymorphism the programmer can have both methods with the same name but they display different things.
- The way we defined those two methods is a technique called **OVERRIDING**.

Inheritance

- Inheritance → Creating new classes from existing ones.
- The child class gets attributes and methods from the parent class
- In the example below:
 - Student inherits the attribute “name” from the parent class which is Person
 - The super keyword is used to call the parent constructor which instantiates the object keeping both classes in mind

Example

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"Name: {self.name}, Age: {self.age}")

class Dog(Animal): # Dog inherits from Animal
    # constructor?
    # info()?

    def bark(self): # something only dogs do...
        print(f"{self.name} says: Woof!")
```

Example

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"Name: {self.name}, Age: {self.age}")

class Dog(Animal): # Dog inherits from Animal
    # constructor?
    # info()?

    def bark(self): # something only dogs do...
        print(f"{self.name} says: Woof!")
```

```
if __name__ == "__main__":
    animal = Animal("Generic Animal", 3)
    animal.info()

    dog = Dog("Buddy", 5)
    dog.info()
    dog.bark()
```

Example

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"Name: {self.name}, Age: {self.age}")

class Dog(Animal): # Dog inherits from Animal
    # constructor?
    # info()?

    def bark(self): # something only dogs do...
        print(f"{self.name} says: Woof!")
```

```
if __name__ == "__main__":
    animal = Animal("Generic Animal", 3)
    animal.info()

    dog = Dog("Buddy", 5)
    dog.info()
    dog.bark()
```

OUTPUT:

```
Name: Generic Animal, Age: 3
Name: Buddy, Age: 5
Buddy says: Woof!
```

Inheritance

Types of inheritance:

- Single inheritance :
A child class inherits from a single parent class (e.g. previous slide)
- Multiple inheritance :
A child class inherits from more than one parent.
- Multi-level inheritance :
A child class inherits from an inherited class
- Hierarchical inheritance :
Multiple child classes inherit from a single parent class
- Hybrid Inheritance :
Combination of two or more of the above

Multiple Inheritance

```
# Parent classes
class Spaceship:
    def launch(self):
        print("Spaceship is launching!")

class WeaponSystem:
    def fire_weapons(self):
        print("Weapons fired!")

# Child class
class BattleShip(Spaceship, WeaponSystem):
    def shield_on(self):
        print("Shields activated!")

# Example usage
bship = BattleShip()
bship.launch()           # From Spaceship
bship.fire_weapons()    # From WeaponSystem
bship.shield_on()       # Own method
```

Multi-level inheritance

```
# Base class
class Spaceship:
    def launch(self):
        print("Spaceship is launching!")

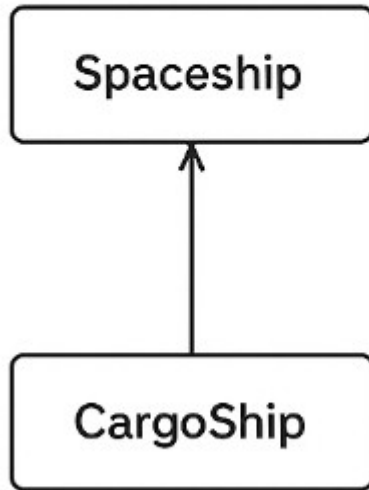
# Derived class
class CargoShip(Spaceship):
    def load_cargo(self):
        print("Cargo is loaded!")

# Further derived class
class InterstellarCargoShip(CargoShip):
    def hyperjump(self):
        print("Hyperjump initiated!")

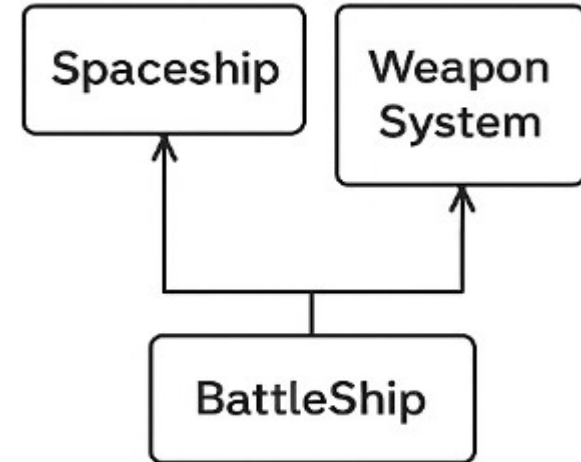
# Example usage
ship = InterstellarCargoShip()
ship.launch()           # From Spaceship
ship.load_cargo()       # From CargoShip
ship.hyperjump()        # Own method
```

UML diagrams

Single Inheritance

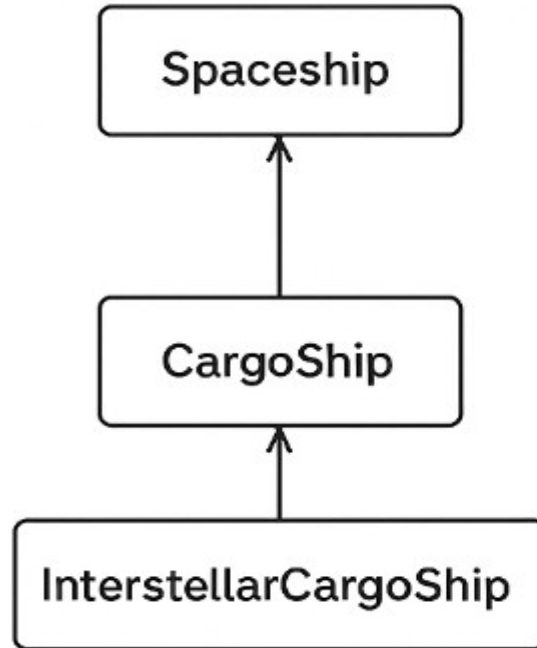


Multiple Inheritance

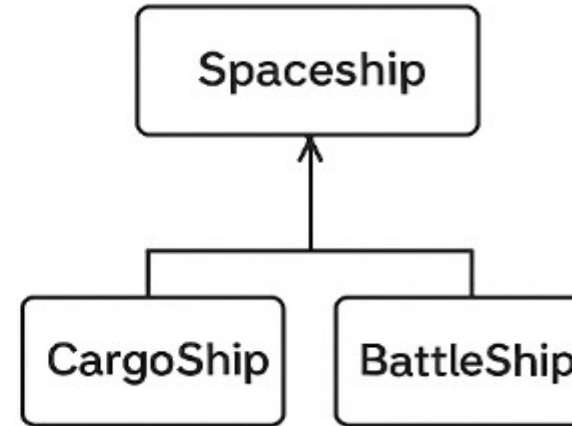


UML diagrams

Multi-Level Inheritance



Hierarchical Inheritance



Abstraction

- Hiding the implementation details and showing only what's necessary.
In Python this is done using abstract classes.
- For this we import ABC, abstractmethod from abc

```
from abc import ABC, abstractmethod

class Animal(ABC):
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"Name: {self.name}, Age: {self.age}")

    @abstractmethod
    def make_sound(self):
        """Each animal must define its own sound."""
        pass

class Dog(Animal):
    def make_sound(self): # mandatory implementation!
        print(f"{self.name} says: Woof!")

    def bark(self):
        # Optional additional behavior
        self.make_sound()
```

Abstraction

- The abstract class can not be instantiated
- The derived classes have to implement @abstractmethod

```
if __name__ == "__main__":
    # animal = Animal("Generic Animal", 3)
    dog = Dog("Buddy", 5)
    dog.info()
    dog.bark()
```

OUTPUT:

```
Name: Buddy, Age: 5
Buddy says: Woof!
```

```
Name: Buddy, Age: 5
Buddy says: Woof!
```

```
from abc import ABC, abstractmethod

class Animal(ABC):
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"Name: {self.name}, Age: {self.age}")

    @abstractmethod
    def make_sound(self):
        """Each animal must define its own sound."""
        pass

class Dog(Animal):
    def make_sound(self): # mandatory implementation!
        print(f"{self.name} says: Woof!")

    def bark(self):
        # Optional additional behavior
        self.make_sound()
```

Abstraction

- As defined before we see that each animal makes its own sound but we don't want to or need to know how it does it.
- Hence we make the sound method of the abstract class Animal an abstract method by using the `@abstractmethod` decorator

Naming Conventions and Practices

- Class names → use PascalCase (Capitalize Each Word)
- Method names → use snake_case (lowercase with underscores)
- One class = one responsibility
 - A class should represent a single concept (Student, Person, Car, Book)

Naming Conventions and Practices

- Attributes = state, Methods = behavior
 - Attributes should hold data, Methods should perform actions
- Avoid making one GIANT class which does everything

Summary

Any questions?